Laboratorio 8 - Teoria de la Computacion

Fatima Navarro, carné 24044 Universidad del Valle de Guatemala

Respuestas a los ejercicios sin código: Problemas 1a, 2a, 3a, 4 y 5.

Problema 1, parte a

```
        void function(int n) {
            int i, j, k, counter = 0;
            for (i = n/2; i <= n; i++) {
                for (j = 1; j+n/2 <= n; j++) {
                    for (k = 1; k <= n; k = k*2) {
                        counter++;
                    }
                }
            }
        }
```

Paso 1: ciclo i. Va desde n/2 hasta n incrementando de 1 en 1.

Iteraciones = n - n/2 + 1 = **n/2 + 1**

Paso 2: ciclo j. La condición es j + n/2 <= n. Despejando j:

j <= n - n/2, o sea j <= n/2

Como j arranca en 1, iteraciones = **n/2**

Es importante notar que el límite del ciclo j no depende de i, entonces el conteo es el mismo en cada vuelta del ciclo externo.

Paso 3: ciclo k. Empieza en k = 1 y en cada vuelta hace k = k*2, así que toma los valores 1, 2, 4, 8, ..., o sea 2^0, 2^1, 2^2, ..., 2^m.

El ciclo se detiene cuando 2^m > n, es decir cuando m > log2(n).

Iteraciones = **floor(log2 n) + 1**

Paso 4: multiplicar. Los tres ciclos están anidados, entonces el total es el producto:

T(n) = (n/2 + 1) · (n/2) · (floor(log2 n) + 1)

T(n) ≈ (n² / 4) · log2(n)

Paso 5: descartar constantes y términos de menor orden.

T(n) = O(n² log n)

Problema 2, parte a

```
        void function(int n) {
            if (n <= 1) return;
            int i, j;
            for (i = 1; i <= n; i++) {
                for (j = 1; j <= n; j++) {
                    printf("Sequence\n");
                    break;
                }
            }
        }
```

Paso 1: caso base. Si n <= 1 la función hace `return` de inmediato, costo O(1).

Paso 2: ciclo j. Aunque la condición dice j <= n, el `break` al final del cuerpo termina el ciclo en su primera iteración, siempre.

Iteraciones = **1**, no n.

Paso 3: ciclo i. Va de 1 a n sin ningún break.

Iteraciones = **n**

Paso 4: multiplicar.

T(n) = n · 1 = n

T(n) = O(n)

Este ejercicio es engañoso a propósito: la forma del código sugiere O(n²) por los dos ciclos anidados, pero el `break` vuelve constante el trabajo del ciclo interno y el algoritmo termina siendo lineal. El análisis se hace sobre lo que el código *ejecuta*, no sobre cómo se ve.

Problema 3, parte a

```
        void function(int n) {
            int i, j;
            for (i = 1; i <= n/3; i++) {
                for (j = 1; j <= n; j += 4) {
                    printf("Sequence\n");
                }
            }
        }
```

Paso 1: ciclo i. Va de 1 hasta n/3 incrementando de 1 en 1.

Iteraciones = **n/3**

Paso 2: ciclo j. Va de 1 hasta n pero con paso 4, entonces toma los valores 1, 5, 9, 13, ...

Iteraciones = **ceil(n/4)**

Paso 3: multiplicar.

T(n) = (n/3) · (n/4) = n² / 12

Paso 4: descartar la constante 1/12.

T(n) = O(n²)

Las constantes 1/3 y 1/4 no cambian la clase de complejidad, pero sí afectan el tiempo real de ejecución: el algoritmo hace 12 veces menos trabajo que uno que recorriera n² completo. Eso se nota en el profiling, donde este problema alcanza n = 100,000 mientras que el Problema 1 se queda en n = 10,000.

Problema 4: mejor caso, caso promedio y peor caso

Búsqueda Lineal (Linear Search)

Recorre el arreglo de principio a fin comparando cada elemento con el buscado.

Mejor caso. El elemento está en la primera posición. Se hace 1 comparación y la función retorna.

T(n) = 1, o sea **O(1)**

Peor caso. El elemento está en la última posición, o no está en el arreglo. En ambos casos hay que recorrer las n posiciones.

T(n) = n, o sea **O(n)**

Caso promedio. Suponiendo que el elemento sí está y que es igualmente probable que esté en cualquiera de las n posiciones, la probabilidad de cada posición es 1/n:

T(n) = (1 + 2 + 3 + ... + n) / n = [n(n+1)/2] / n = (n+1)/2

Descartando la constante 1/2: **O(n)**

Búsqueda Binaria (Binary Search)

Requiere que el arreglo esté ordenado. Compara con el elemento del medio y descarta la mitad que no puede contener el buscado.

Mejor caso. El elemento está justo en la posición del medio. Se hace 1 comparación.

T(n) = 1, o sea **O(1)**

Peor caso. El elemento no está, o está en una hoja del árbol de búsqueda. Hay que dividir hasta quedarse con un solo elemento.

Después de k divisiones quedan n / 2^k elementos. El proceso termina cuando:

n / 2^k = 1 → 2^k = n → k = log2(n)

T(n) = floor(log2 n) + 1, o sea **O(log n)**

Caso promedio. En el árbol de búsqueda binaria de n elementos, la mitad de los elementos está en el último nivel, y aproximadamente 3/4 están en los últimos dos niveles. Es decir, la mayoría de los casos cae cerca de la profundidad máxima, y el promedio es log2(n) - 1 comparaciones aproximadamente.

O(log n)

Quicksort

Elige un pivote, particiona el arreglo en elementos menores y mayores al pivote (costo lineal, n comparaciones), y se llama recursivamente sobre cada parte.

Mejor caso. El pivote siempre resulta ser la mediana, así que parte el arreglo en dos mitades iguales.

$T(n) = 2 \cdot T(n/2) + n$

Aplicando el teorema maestro con a = 2, b = 2, f(n) = n:

$n^{(\log_b a)} = n^{(\log_2 2)} = n^1 = n = f(n)$, entonces estamos en el caso 2:

$T(n) = \mathbf{O(n \log n)}$

Peor caso. El pivote siempre resulta ser el elemento más pequeño o el más grande, así que una partición queda vacía y la otra con n-1 elementos. Esto pasa, por ejemplo, con un arreglo ya ordenado si se toma siempre el primer elemento como pivote.

$T(n) = T(n-1) + n$

Desenrollando la recurrencia:

$T(n) = n + (n-1) + (n-2) + \ldots + 2 + 1 = n(n+1)/2$

$T(n) = \mathbf{O(n^2)}$

Caso promedio. Promediando sobre todas las n! permutaciones posibles del arreglo de entrada, la partición es lo suficientemente balanceada la mayoría de las veces. La recurrencia promedio da:

$T(n) = \mathbf{O(n \log n)}$

En la práctica, escoger el pivote de forma aleatoria o con la técnica de mediana de tres hace que el peor caso sea extremadamente improbable.

Resumen

Algoritmo	Mejor caso	Caso promedio	Peor caso
Búsqueda Lineal	O(1)	O(n)	O(n)
Búsqueda Binaria	O(1)	O(log n)	O(log n)
Quicksort	O(n log n)	O(n log n)	$O(n^2)$

Problema 5: verdadero o falso

a) Si $f(n) = \Theta(g(n))$ y $g(n) = \Theta(h(n))$, entonces $h(n) = \Theta(f(n))$

VERDADERO.

Θ es una relación de equivalencia, es decir que cumple las tres propiedades: reflexiva, simétrica y transitiva.

Por **transitividad**: como f = Θ(g) y g = Θ(h), entonces f = Θ(h).

Por **simetría**: si f = Θ(h), entonces h = Θ(f).

Demostración con la definición: f = Θ(g) significa que existen constantes positivas c1, c2 y n0 tales que

c1 · g(n) <= f(n) <= c2 · g(n) para todo n >= n0

y g = Θ(h) significa que existen d1, d2 y n1 tales que

d1 · h(n) <= g(n) <= d2 · h(n) para todo n >= n1

Sustituyendo, para todo n >= max(n0, n1):

c1 · d1 · h(n) <= f(n) <= c2 · d2 · h(n)

Tomando las constantes (c1·d1) y (c2·d2) queda f = Θ(h), y despejando h:

(1 / (c2·d2)) · f(n) <= h(n) <= (1 / (c1·d1)) · f(n)

que es exactamente **h = Θ(f)**.

b) Si f(n) = O(g(n)) y g(n) = O(h(n)), entonces h(n) = Ω(f(n))

VERDADERO.

Por **transitividad de O**: f = O(g) y g = O(h) implica f = O(h).

Demostración: f = O(g) significa que existen c y n0 con f(n) <= c · g(n) para n >= n0. Y g = O(h) significa que existen d y n1 con g(n) <= d · h(n) para n >= n1.

Encadenando, para todo n >= max(n0, n1):

f(n) <= c · g(n) <= c · d · h(n)

Entonces f = O(h) con constante c·d.

Ahora, la relación entre O y Ω: por definición, f = O(h) si y solo si h = Ω(f). La desigualdad f(n) <= (c·d) · h(n) se puede reescribir como

h(n) >= (1 / (c·d)) · f(n)

que es la definición de **h = Ω(f)** con constante 1/(c·d).

c) f(n) = Θ(n²), donde f(n) es el tiempo de ejecución del programa

```
        def A(n):
            atupla = tuple(range(0, n))
            S = set()
            for i in range(0, n):
                for j in range(i + 1, n):
                    S.add(atupla[i:j])
```

FALSO. La complejidad real es Θ(n³).

Paso 1: contar las iteraciones. Los dos ciclos recorren todos los pares (i, j) con i < j. La cantidad de pares es:

C(n, 2) = n(n-1)/2 ≈ n² / 2

Si cada iteración costara O(1), la respuesta sería Θ(n²) y el enunciado sería verdadero. Pero no es así.

Paso 2: el costo real de cada iteración. La operación `atupla[i:j]` es un slice, que **crea una tupla nueva** copiando j - i elementos. Eso cuesta Θ(j - i), no O(1).

Además, `S.add(...)` tiene que calcular el hash de esa tupla, y hashear una tupla requiere recorrer sus j - i elementos. Eso es otro Θ(j - i).

Entonces el costo de cada iteración es Θ(j - i), proporcional al largo del slice.

Paso 3: sumar sobre todas las iteraciones.

T(n) = suma sobre i, con j de i+1 a n-1, de (j - i)

Conviene cambiar de variable. Sea d = j - i el largo del slice. Para cada valor de d entre 1 y n-1, la cantidad de pares (i, j) con j - i = d es exactamente (n - d).

T(n) = suma desde d=1 hasta n-1 de [d · (n - d)]

Separando la suma:

T(n) = n · (suma de d) - (suma de d²)

T(n) = n · [(n-1)n / 2] - [(n-1)n(2n-1) / 6]

Factorizando (n-1)n:

T(n) = (n-1)n · [n/2 - (2n-1)/6]

T(n) = (n-1)n · [(3n - 2n + 1) / 6]

T(n) = (n-1)n(n+1) / 6

T(n) = (n³ - n) / 6

Paso 4: conclusión.

T(n) = Θ(n³), no Θ(n²). El enunciado es FALSO.

Este ejercicio ilustra un error muy común: asumir que toda operación dentro de un ciclo cuesta O(1). En Python, los slices, la concatenación de strings y el hashing de estructuras son operaciones lineales en el tamaño del dato, no constantes.